

# Primary sources & further reading

Part 06 — 🧠 AI Agents · 06-ai-agents/papers/PAPERS.md · 06-ai-agents/resources/RESOURCES.md



## Papers & Primary Sources — AI Agents

arXiv links are canonical and stable. Read the three starred papers in order before anything else — together they are the entire intellectual history of "give an LLM tools and a loop," and almost everything since is a variation on one of them.

### The canon (read in this order)

| # | Paper                                                                                  | Year | Why it matters                                                                                                                                                                      | Link                             |
|---|----------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 1 | ★ <b>ReAct: Synergizing Reasoning and Acting in Language Models</b> — Yao et al.       | 2022 | The pattern this entire module is built around. Read the prompts in Appendix A/B — they are <code>code/react_loop_from_scratch.py</code> , in the original.                         | <a href="#">arXiv:2210.03629</a> |
| 2 | ★ <b>Toolformer: Language Models Can Teach Themselves to Use Tools</b> — Schick et al. | 2023 | Shows a model can learn <i>when</i> to call a tool from self-supervised examples, rather than always from a hand-written prompt. The conceptual root of native tool-calling APIs.   | <a href="#">arXiv:2302.04761</a> |
| 3 | ★ <b>Reflexion: Language Agents with Verbal Reinforcement Learning</b> — Shinn et al.  | 2023 | Self-critique as memory: a verbal reflection on a failed attempt, stored and consulted on the next one. The clearest implementation of procedural memory in this module's taxonomy. | <a href="#">arXiv:2303.11366</a> |

## Planning & search-based reasoning

| Paper                                                                                       | Year | One-line takeaway                                                                                                                                                               | Link                             |
|---------------------------------------------------------------------------------------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>Chain-of-Thought Prompting Elicits Reasoning in Large Language Models</b> — Wei et al.   | 2022 | The reasoning half of ReAct, in isolation — read this to see exactly what "acting" adds on top.                                                                                 | <a href="#">arXiv:2201.11903</a> |
| <b>Tree of Thoughts: Deliberate Problem Solving with Large Language Models</b> — Yao et al. | 2023 | Branch, evaluate, backtrack — search over reasoning paths instead of committing to one. Expensive; use only where the task is genuinely combinatorial.                          | <a href="#">arXiv:2305.10601</a> |
| <b>Plan-and-Solve Prompting</b> — Wang et al.                                               | 2023 | Explicit upfront decomposition before execution, as a prompting strategy rather than a framework.                                                                               | <a href="#">arXiv:2305.04091</a> |
| <b>ReWOO: Decoupling Reasoning from Observations</b> — Xu et al.                            | 2023 | Plans the whole tool sequence before executing any of it, cutting redundant model calls versus interleaved ReAct — the efficiency argument against always re-planning per step. | <a href="#">arXiv:2305.18323</a> |
| <b>Self-Refine: Iterative Refinement with Self-Feedback</b> — Madaan et al.                 | 2023 | The single-pass sibling of Reflexion: critique and revise within one episode, no persisted memory required.                                                                     | <a href="#">arXiv:2303.17651</a> |

## Tool use & orchestration

| Paper                                                                           | Year | One-line takeaway                                                                                                                                    | Link                             |
|---------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>MRKL Systems</b> — Karpas et al.                                             | 2022 | Modular Reasoning, Knowledge and Language: routing between an LLM and external "expert modules" — the architectural ancestor of today's tool router. | <a href="#">arXiv:2205.00445</a> |
| <b>WebGPT: Browser-assisted Question-Answering</b> — Nakano et al.              | 2021 | An early, concrete agent: search, click, cite — before "agent" was the word anyone used.                                                             | <a href="#">arXiv:2112.09332</a> |
| <b>Gorilla: Large Language Model Connected with Massive APIs</b> — Patil et al. | 2023 | What happens to tool-selection accuracy as the tool catalog grows into the thousands — direct evidence for notes/02 §2's "too many tools" claim.     | <a href="#">arXiv:2305.15334</a> |
| <b>HuggingGPT</b> — Shen et al.                                                 | 2023 | An LLM as a controller dispatching to specialist models as tools — task decomposition made literal.                                                  | <a href="#">arXiv:2303.17580</a> |

## Memory & long-horizon agents

| Paper                                                                                 | Year | One-line takeaway                                                                                                                                                                                      | Link                             |
|---------------------------------------------------------------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>Generative Agents: Interactive Simulacra of Human Behavior</b> — Park et al.       | 2023 | The "memory stream" architecture — episodic memory with retrieval, reflection, and planning, in a simulated town. The clearest worked example of episodic memory as an implementation, not a metaphor. | <a href="#">arXiv:2304.03442</a> |
| <b>Voyager: An Open-Ended Embodied Agent with Large Language Models</b> — Wang et al. | 2023 | A Minecraft agent that writes and saves reusable skills as code — procedural memory that actually accumulates across episodes.                                                                         | <a href="#">arXiv:2305.16291</a> |
| <b>A Survey on Large Language Model based Autonomous Agents</b> — Wang et al.         | 2023 | The field-wide map: construction, application, evaluation. Useful after you've read the primary papers above, not before.                                                                              | <a href="#">arXiv:2308.11432</a> |

## Official docs (the mechanics, from the source)

| Source             | What it covers                                                                                                        | Link                                                                                                                            |
|--------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Anthropic docs     | Tool use / function calling: schema format, the request/response round trip, parallel tool calls, error handling      | <a href="https://docs.claude.com/">https://docs.claude.com/</a>                                                                 |
| Anthropic Cookbook | Runnable tool-use and agent patterns beyond the docs' minimal examples                                                | <a href="https://github.com/anthropics/anthropic-cookbook">https://github.com/anthropics/anthropic-cookbook</a>                 |
| OpenAI docs        | Function calling: the same mechanics from a second vendor, useful for spotting what's universal vs. provider-specific | <a href="https://platform.openai.com/docs/guides/function-calling">https://platform.openai.com/docs/guides/function-calling</a> |

## How to read an agent paper (30 minutes, 3 passes)

1. **Pass 1 (5 min)**: abstract, the main diagram (almost always a loop or a tree), and the headline benchmark number. What's the claimed capability, and on what task family?
2. **Pass 2 (15 min)**: find the actual prompt in the appendix. Agent papers live or die on prompt engineering that the abstract never mentions — read the literal Thought/Action text before believing the architecture diagram.
3. **Pass 3 (10 min)**: the failure analysis, if there is one (not all agent papers have one — treat that absence as a yellow flag). Which failure modes from [notes/02 §3](#) does this paper's method actually address, and which does it just not encounter in its benchmark?

Agent benchmarks are notoriously easy to overfit to (a fixed tool set, a narrow task distribution) — the mechanism in a paper generalizes far more reliably than its reported success rate does.

## Resources — AI Agents

### Courses & lectures (free)

| Resource                                                            | Why                                                                                                                                                                  | Link                                                                                                                                                                  |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LLM Agents MOOC (Berkeley)</b>                                   | The most rigorous free course on this exact material — reasoning, planning, tool use, memory, evaluation, taught by the people who wrote several of the canon papers | <a href="https://llmagents-learning.org/f24">https://llmagents-learning.org/f24</a>                                                                                   |
| <b>DeepLearning.AI — Functions, Tools and Agents with LangChain</b> | Short, hands-on: the tool-calling round trip and the ReAct loop, built incrementally                                                                                 | <a href="https://www.deeplearning.ai/short-courses/functions-tools-agents-langchain/">https://www.deeplearning.ai/short-courses/functions-tools-agents-langchain/</a> |
| <b>DeepLearning.AI — AI Agents in LangGraph</b>                     | The runtime side of this module — pairs directly with <code>../.. /16-ai-tech-stack/tracks/langgraph/</code>                                                         | <a href="https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/">https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/</a>                     |
| <b>Prompting Guide — LLM Agents</b>                                 | A concise, well-maintained reference on ReAct, planning and agent architectures, with links back to the primary papers                                               | <a href="https://www.promptingguide.ai/research/llm-agents">https://www.promptingguide.ai/research/llm-agents</a>                                                     |

### Repositories worth cloning

| Repo                                                                                                                    | What's inside                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <a href="https://github.com/ysymyth/ReAct">https://github.com/ysymyth/ReAct</a>                                         | The original ReAct paper's code and prompts — read the actual few-shot exemplars                                             |
| <a href="https://github.com/noahshinn/reflexion">https://github.com/noahshinn/reflexion</a>                             | Reference implementation of Reflexion's verbal self-critique loop                                                            |
| <a href="https://github.com/princeton-nlp/tree-of-thought-llm">https://github.com/princeton-nlp/tree-of-thought-llm</a> | Tree of Thoughts reference implementation, including the scoring/pruning logic                                               |
| <a href="https://github.com/ShishirPatil/gorilla">https://github.com/ShishirPatil/gorilla</a>                           | Large-scale API/tool-calling benchmark and models — the "too many tools" evidence in code form                               |
| <a href="https://github.com/anthropics/anthropic-cookbook">https://github.com/anthropics/anthropic-cookbook</a>         | Runnable tool-use and agent recipes against the Claude API                                                                   |
| <a href="https://github.com/langchain-ai/langchain">https://github.com/langchain-ai/langchain</a>                       | The tool/model abstraction layer most agent frameworks build on — see <code>../.. /16-ai-tech-stack/tracks/langchain/</code> |
| <a href="https://github.com/langchain-ai/langgraph">https://github.com/langchain-ai/langgraph</a>                       | The durable, resumable agent-loop runtime — see <code>../.. /16-ai-tech-stack/tracks/langgraph/</code>                       |

### Cheat sheets

- Anthropic docs (tool use, agent SDK, agent design patterns): <https://docs.claude.com/>
- OpenAI function calling guide: <https://platform.openai.com/docs/guides/function-calling>
- Also see `../.. /_shared/cheatsheets/`

## Where the theory meets production (this repo)

| Concern                                                                  | Where                                                         |
|--------------------------------------------------------------------------|---------------------------------------------------------------|
| Durable, resumable agent runtime; cycles, checkpoints, human-in-the-loop | <code>../../../../16-ai-tech-stack/tracks/langgraph/</code>   |
| The model/tool abstraction layer, concrete implementation                | <code>../../../../16-ai-tech-stack/tracks/langchain/</code>   |
| A real agent harness applying these exact recovery/permission patterns   | <code>../../../../16-ai-tech-stack/tracks/claude-code/</code> |
| Multi-agent topologies, hand-offs, supervisors                           | Module 07 — Agentic AI Systems [07-agentic-ai]                |
| The lethal trifecta, prompt injection, tool-access abuse                 | Module 13 — AI Security [13-ai-security]                      |
| Trajectory grading, agent eval harnesses                                 | Module 15 — AI Evaluation [15-ai-evals]                       |
| Standardizing tool/data access across agents                             | Module 09 — MCP [09-mcp]                                      |

## Communities

- r/LocalLLaMA and r/LangChain — active discussion of what actually breaks in agent deployments, not just demos
- LangChain / LangGraph Discord — the people building the runtime this module's theory feeds into
- Hugging Face forums (`agents` tag) — <https://discuss.huggingface.co/>